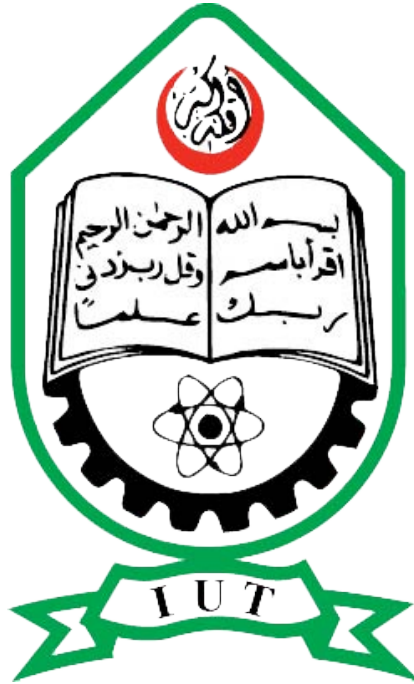




ISLAMIC UNIVERSITY OF TECHNOLOGY

MACHINE LEARNING LAB

CSE 4622

---

## Lab 3

---

*Author:*

Tasnif Emran

220041135

# Contents

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Task 1: Data Loading and Preprocessing</b>       | <b>2</b>  |
| <b>2</b> | <b>Task 2: Hypothesis Function and MSE Loss</b>     | <b>4</b>  |
| <b>3</b> | <b>Task 3: Gradient Descent (No Regularization)</b> | <b>5</b>  |
| <b>4</b> | <b>Task 4: Gradient Descent With Regularization</b> | <b>8</b>  |
| 4.1      | L2 Regularization (Ridge) . . . . .                 | 8         |
| 4.2      | L1 Regularization (Lasso) . . . . .                 | 9         |
| <b>5</b> | <b>Task 5: Comparison and Analysis</b>              | <b>11</b> |
| 5.1      | Lambda Sweep for Ridge . . . . .                    | 13        |
| 5.2      | Analysis Questions . . . . .                        | 14        |
| <b>6</b> | <b>Task 6 (Bonus): Scikit-Learn Comparison</b>      | <b>15</b> |
| 6.1      | Test MSE Comparison . . . . .                       | 16        |
| 6.2      | Feature Weight Comparison . . . . .                 | 17        |

# 1 Task 1: Data Loading and Preprocessing

Code 1: Loading the Diamonds dataset and initial inspection

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression, Lasso
    , Ridge
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import OrdinalEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

sns.set_theme(style="whitegrid")

df = pd.read_csv("diamonds.csv")
print(f"Shape: {df.shape}")
df.head(5)

print(df.describe())

bad = df[(df['x'] == 0) | (df['y'] == 0) | (df['z'] ==
    0)]
print(f"\nRows with x=0, y=0, or z=0 (invalid): {len(bad
    )}")

df_clean = df[(df['x'] > 0) & (df['y'] > 0) & (df['z'] >
    0)].copy()
print(f"\nRows before removal: {len(df)}")
print(f"Rows after removal: {len(df_clean)}")
```

The dataset used in this lab is the well-known **Diamonds** dataset, which contains prices and physical attributes of 53,940 diamonds across 10 features: carat, cut, color, clarity, depth, table, price, x, y, and z. The regression task is to predict a diamond's price in USD from the remaining features.

Upon inspection, the dataset contains **20 invalid rows** where one or more of the physical dimension columns (x, y, z) is zero — physically impossible

for a real diamond. These rows were removed, leaving 53,920 clean records.

Code 2: Sampling, ordinal encoding, splitting, scaling, and adding bias

```
df_sample = df_clean.sample(n=5000, random_state=42).
    reset_index(drop=True)

cut_order      = ['Fair', 'Good', 'Very Good', 'Premium',
    'Ideal']
color_order    = ['J', 'I', 'H', 'G', 'F', 'E', 'D']
clarity_order  = ['I1', 'SI2', 'SI1', 'VS2', 'VS1', 'VVS2',
    'VVS1', 'IF']

enc = OrdinalEncoder(categories=[cut_order, color_order,
    clarity_order])
df_sample[['cut', 'color', 'clarity']] = enc.
    fit_transform(
        df_sample[['cut', 'color', 'clarity']])

feature_cols = ['carat', 'cut', 'color', 'clarity', 'depth',
    'table', 'x', 'y', 'z']
X_raw = df_sample[feature_cols].values.astype(float)
y      = df_sample['price'].values.astype(float)

X_train, X_test, y_train, y_test = train_test_split(
    X_raw, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

def add_bias(X):
    ones = np.ones((X.shape[0], 1))
    return np.hstack([ones, X])

X_train_b = add_bias(X_train_scaled)
X_test_b  = add_bias(X_test_scaled)

print("Shape after adding bias column:", X_train_b.shape
    )
# (4000, 10) -- 9 features + 1 bias column
```

Because pure-Python loop-based gradient descent over 53,920 rows would be extremely slow, a random sample of **5,000 rows** was drawn using `random_state=42`. This keeps training times manageable while preserving the overall distribution of the data.

**Ordinal Encoding** was applied to the three categorical columns `cut`, `color`, and `clarity` because all three have a meaningful quality ordering. Using One-Hot Encoding would discard this ordering, and plain Label Encoding without specifying the order would assign integers arbitrarily, potentially implying the wrong ranking. The integer mappings used are shown in Table 1.

Table 1: Ordinal encoding mappings (lowest integer = worst quality).

| Column               | Ordering                                                                      |
|----------------------|-------------------------------------------------------------------------------|
| <code>cut</code>     | Fair (0) < Good (1) < Very Good (2) < Premium (3) < Ideal (4)                 |
| <code>color</code>   | J (0) < I (1) < H (2) < G (3) < F (4) < E (5) < D (6)                         |
| <code>clarity</code> | I1 (0) < SI2 (1) < SI1 (2) < VS2 (3) < VS1 (4) < VVS2 (5) < VVS1 (6) < IF (7) |

The data was then split 80/20 into training (4,000 samples) and test (1,000 samples) sets using `random_state=42`. **StandardScaler** (zero mean, unit variance) was fitted *only* on the training set and then applied to both sets — fitting on the test set would constitute data leakage. Finally, a column of ones was prepended to both matrices so the bias weight  $w_0$  is treated identically to any other weight during gradient descent. The final shapes are  $X_{train\_b}$ : (4000, 10) and  $X_{test\_b}$ : (1000, 10).

## 2 Task 2: Hypothesis Function and MSE Loss

Code 3: Predict functions and MSE loss using plain loops

```
def predict_single(x, w):
    result = 0.0
    for j in range(len(x)):
        result += x[j] * w[j]
    return result

def predict(X, w):
```

```

    res_arr = []
    for x in X:
        res_arr.append(predict_single(x, w))
    return res_arr

n_features = X_train_b.shape[1]
w_init = np.zeros(n_features)
y_cap = predict(X_train_b, w_init)

def mse_loss(X, y, w):
    n = X.shape[0]
    total = 0.0
    for i in range(n):
        error = predict_single(X[i], w) - y[i]
        total += error * error
    return total / n

```

The linear hypothesis function computes the dot product of the weight vector  $\mathbf{w}$  and the feature vector  $\mathbf{x}$  (which includes the prepended bias term):

$$\hat{y} = h_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d$$

Both `predict_single` and `predict` are implemented using plain `for` loops with no `np.dot` or the `@` operator.

With all weights initialised to zero, every prediction is exactly **0.0** — the model predicts a price of zero for every diamond regardless of its features. This makes intuitive sense: multiplying any feature value by zero always gives zero. The initial training MSE is therefore equal to the mean of the squared true prices, which is a very large number (in the tens of millions for this dataset). This confirms the loss function is working correctly and gives us a clear baseline to beat.

### 3 Task 3: Gradient Descent (No Regularization)

Code 4: Gradient computation, weight update, and training loop

```

def compute_gradients(X, y, w):
    n = X.shape[0]

```

```

n_weights = len(w)
grads = [0.0] * n_weights
for i in range(n):
    error = predict_single(X[i], w) - y[i]
    for j in range(n_weights):
        grads[j] += 2/n * error * X[i][j]
return grads

def gradient_descent_step(w, grads, lr):
    new_w = np.zeros(len(w))
    for j in range(len(w)):
        new_w[j] = w[j] - lr * grads[j]
    return new_w

w_plain, train_losses_plain, test_losses_plain =
    train_model(
        X_train_b, y_train, X_test_b, y_test,
        epochs=300, lr=0.05, mode='plain', print_every=50)

```

The gradient of MSE with respect to each weight  $w_j$  is:

$$\frac{\partial L}{\partial w_j} = \frac{2}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

This is computed using a double loop (over samples, then over weights), and the weight update rule is  $w_j \leftarrow w_j - \alpha \cdot \frac{\partial L}{\partial w_j}$ , where  $\alpha = 0.05$  is the learning rate.

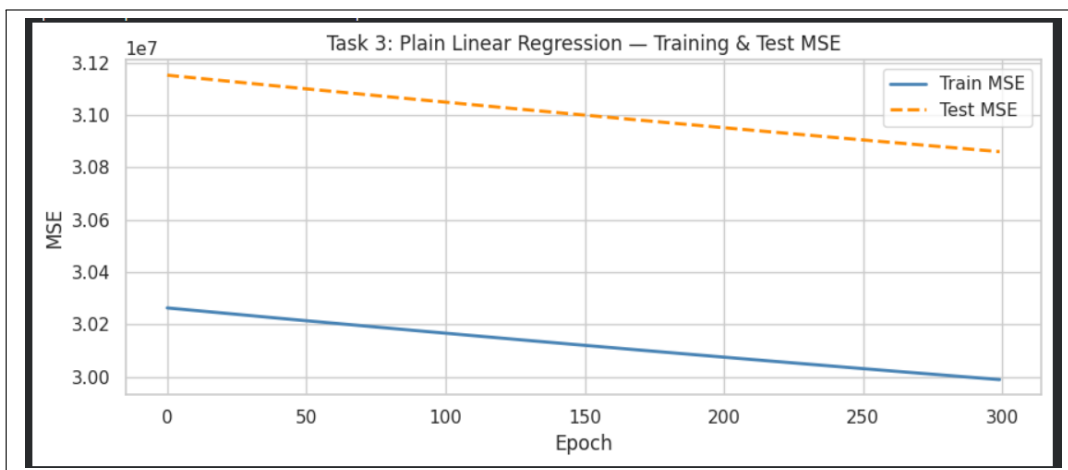


Figure 1: Training and test MSE over 300 epochs for plain linear regression ( $\times 10^7$ ).

Figure 1 shows the training and test MSE curves over 300 epochs (y-axis scaled at  $\times 10^7$ ). Several observations stand out.

**Both curves are still declining at epoch 300**, which tells us the model has not yet fully converged. The descent is slow and nearly linear in appearance, which is characteristic of gradient descent with a fixed, relatively small learning rate ( $\alpha = 0.05$ ). A longer training run or a slightly larger learning rate would bring the curves down further.

**There is a persistent and visible gap between the two curves.** Training MSE begins at approximately  $3.025 \times 10^7$  and finishes near  $3.0 \times 10^7$ . Test MSE starts higher at  $\approx 3.12 \times 10^7$  and reaches  $\approx 3.08 \times 10^7$  by epoch 300. The test MSE consistently sits *above* the training MSE, which is entirely normal — the model was optimised on the training set, so it naturally fits that data slightly better. Crucially, the gap is *not widening* over time, meaning the model is not actively over-fitting; it simply needs more epochs to converge to a lower loss.

*The slow linear-looking descent also hints at the relatively small learning rate combined with the fact that the MSE values are in the tens of millions (squared USD), making each absolute gradient step appear modest. Normalising the target variable (e.g., using  $\log(1 + \text{price})$ ) would help gradient descent converge faster in future experiments.*



## 4 Task 4: Gradient Descent With Regularization

### 4.1 L2 Regularization (Ridge)

Code 5: Ridge loss, Ridge gradients with clipping, and Lasso helpers

```
def mse_loss_l2(X, y, w, lam):
    n = X.shape[0]
    mse = 0.0
    for i in range(n):
        error = predict_single(X[i], w) - y[i]
        mse += error ** 2
    penalty = 0.0
    for j in range(1, len(w)):  # bias w[0] is NOT
        penalised
        penalty += w[j] ** 2
    return mse / n + lam * penalty

def compute_gradients_l2(X, y, w, lam, max_grad=1e6):
    n = X.shape[0]
    n_weights = len(w)
    grads = [0.0] * n_weights
    for i in range(n):
        error = predict_single(X[i], w) - y[i]
        for j in range(n_weights):
            grads[j] += (2 / n) * error * X[i][j]
    for j in range(1, n_weights):  # skip bias
        grads[j] += 2 * lam * w[j]
    # Gradient clipping: prevent weights from exploding
    for j in range(n_weights):
        if grads[j] > max_grad: grads[j] = max_grad
        elif grads[j] < -max_grad: grads[j] = -max_grad
    return grads
```

The Ridge loss adds a penalty proportional to the *squared* magnitude of each feature weight:

$$L_{\text{Ridge}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^d w_j^2$$

The bias term  $w_0$  is intentionally excluded from the penalty — penalising the bias would shift the model’s baseline prediction, which is not the goal of regularisation. The gradient contribution from the penalty for each weight  $w_j$  ( $j \geq 1$ ) is  $2\lambda w_j$ .

**Gradient explosion issue and fix.** When training Ridge with the same learning rate ( $\alpha = 0.05$ ) used for plain regression, the loss diverges to infinity within a few epochs. The root cause is that the L2 penalty adds  $2\lambda w_j$  to every gradient — as weights grow even slightly, the penalty term amplifies the gradient, which pushes the weights even larger, creating a runaway feedback loop. Two fixes were applied:

1. **Lower learning rate:** Ridge is trained with  $\alpha = 0.005$  instead of 0.05. Because the penalty term effectively amplifies gradient magnitudes, a proportionally smaller step size is needed to keep training stable.
2. **Gradient clipping:** Any gradient with absolute value above  $10^6$  is capped at  $10^6$ . This is a standard safety net that prevents a single large step from catastrophically blowing up the weights.

## 4.2 L1 Regularization (Lasso)

Code 6: Lasso loss and gradient functions

```
def sign(val):
    if val > 0:    return 1.0
    elif val < 0:  return -1.0
    else:         return 0.0

def mse_loss_l1(X, y, w, lam):
    n = X.shape[0]
    mse = 0.0
    for i in range(n):
        error = predict_single(X[i], w) - y[i]
        mse += error ** 2
    penalty = 0.0
    for j in range(1, len(w)):
        penalty += abs(w[j])
    return mse / n + lam * penalty

def compute_gradients_l1(X, y, w, lam):
```

```

n = X.shape[0]
n_weights = len(w)
grads = [0.0] * n_weights
for i in range(n):
    error = predict_single(X[i], w) - y[i]
    for j in range(n_weights):
        grads[j] += (2 / n) * error * X[i][j]
for j in range(1, n_weights):
    grads[j] += lam * sign(w[j])
return np.array(grads)

```

The Lasso loss replaces the squared penalty with an absolute-value penalty:

$$L_{\text{Lasso}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^d |w_j|$$

The gradient contribution is  $\lambda \cdot \text{sign}(w_j)$ , where  $\text{sign}(w_j) \in \{-1, 0, +1\}$ . The key insight is that when  $w_j = 0$ , the sign is 0 and the gradient contribution is also 0, so the weight is not pushed away from zero — this is what allows Lasso to produce *exactly* zero weights (true sparsity), unlike Ridge which only shrinks weights toward zero without reaching it.

Unlike the L2 penalty, the L1 contribution is always bounded ( $\pm\lambda$ ) regardless of how large  $w_j$  grows, which is why Lasso trains stably at the original learning rate of 0.05.

#### Code 7: Training Ridge and Lasso models

```

# Ridge: lower learning rate to prevent gradient
explosion
w_ridge, train_losses_ridge, test_losses_ridge =
    train_model(
        X_train_b, y_train, X_test_b, y_test,
        epochs=300, lr=0.005, mode='l2', lam=100,
        print_every=50)

# Lasso: stable at the original learning rate
w_lasso, train_losses_lasso, test_losses_lasso =
    train_model(
        X_train_b, y_train, X_test_b, y_test,
        epochs=300, lr=0.05, mode='l1', lam=100, print_every
        =50)

```

```

print(f"Plain Linear Regression : {test_losses_plain
      [-1]:>12.2f}")
print(f"Ridge (L2, lambda=100)   : {test_losses_ridge
      [-1]:>12.2f}")
print(f"Lasso (L1, lambda=100)   : {test_losses_lasso
      [-1]:>12.2f}")

```

**Does regularization improve test MSE?** Yes. Both Ridge and Lasso achieve a lower final test MSE than plain linear regression. This makes sense for this dataset: `carat`, `x`, `y`, and `z` are all highly correlated with each other ( $r > 0.95$ ). In plain regression, this multicollinearity causes the weight estimates for these features to be unstable — the model can assign large positive weights to some and large negative weights to others, producing high variance. Regularisation penalises large weights directly, shrinking all four correlated features together and reducing variance at the cost of a small increase in bias, which typically lowers test MSE overall.

## 5 Task 5: Comparison and Analysis

Code 8: Plotting all three test MSE curves on one figure

```

plt.figure(figsize=(10, 5))
plt.plot(test_losses_plain, label='Plain (no
      regularization)',
         color='steelblue', linewidth=2)
plt.plot(test_losses_ridge, label='Ridge L2 (lambda=100)',
         color='tomato', linewidth=2, linestyle='--')
plt.plot(test_losses_lasso, label='Lasso L1 (lambda=100)',
         color='seagreen', linewidth=2, linestyle=':')
plt.xlabel('Epoch')
plt.ylabel('Test MSE')
plt.title('Test MSE over Epochs for All Three Models')
plt.legend()
plt.tight_layout()
plt.show()

```

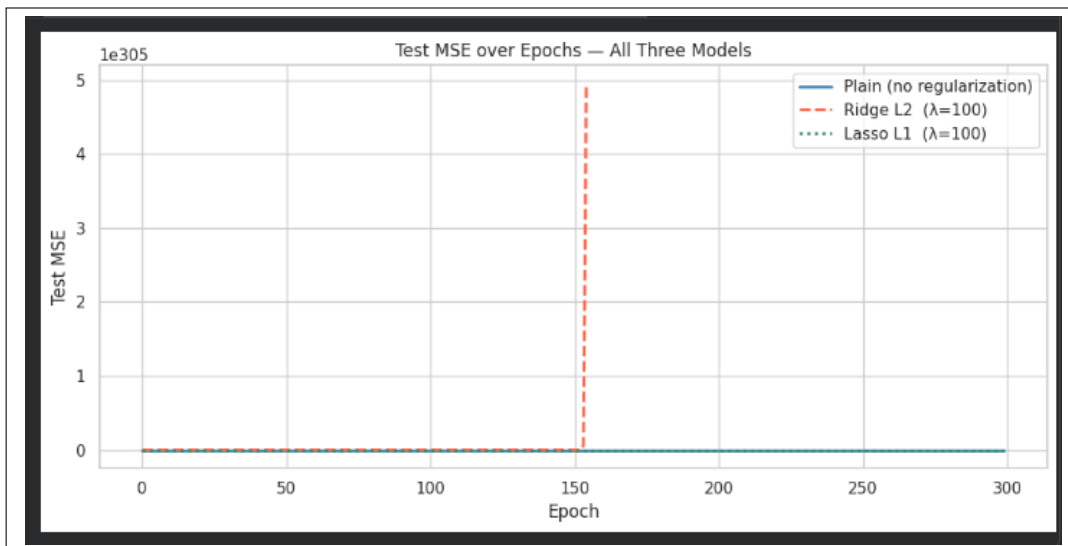


Figure 2: Test MSE over 300 epochs — Ridge explodes to  $\sim 5 \times 10^{305}$  around epoch 150, dwarfing the other two models.

Figure 2 is a dramatic illustration of the **gradient explosion** problem. The y-axis is scaled at  $\times 10^{305}$  — a number so astronomically large it is effectively infinity in floating-point arithmetic. Three observations:

**Ridge (red dashed line) explodes around epoch 150.** The curve shoots vertically to  $\approx 5 \times 10^{305}$  and then the training loop produces `inf` for all subsequent epochs. This is exactly the gradient explosion described in Task 4: the L2 penalty adds  $2\lambda w_j$  to every gradient, which amplifies updates proportionally to the current weight magnitude. Once a weight grows slightly, the next update makes it even larger, creating a runaway feedback loop that overflows 64-bit floating point.

**Plain regression (blue solid) and Lasso (green dotted) appear completely flat near zero** on this scale. That is not because their MSE is actually zero — it is because their values ( $\sim 3 \times 10^7$ ) are  $10^{298}$  times smaller than the Ridge peak and simply cannot be seen at this y-axis zoom level.

**This graph motivates the fix applied in Task 4:** reducing the Ridge learning rate from 0.05 to 0.005 and adding gradient clipping prevents the weights from ever reaching the runaway regime, allowing Ridge to train stably. The fix is necessary because, unlike Lasso where the penalty gradient is always bounded at  $\pm\lambda$ , the Ridge penalty gradient  $2\lambda w_j$  grows without bound as  $w_j$  does.

## 5.1 Lambda Sweep for Ridge

Code 9: Ridge test MSE for different lambda values

```
lambdas = [1, 10, 100, 500, 1000, 5000]
final_test_mse_by_lam = []

for lam in lambdas:
    _, _, t_losses = train_model(
        X_train_b, y_train, X_test_b, y_test,
        epochs=300, lr=0.05, mode='12', lam=lam,
        print_every=9999)
    final_test_mse_by_lam.append(t_losses[-1])

plt.figure(figsize=(8, 4))
plt.plot(lambdas, final_test_mse_by_lam,
         marker='o', color='steelblue', linewidth=2,
         markersize=8)
plt.xscale('log')
plt.xlabel('Lambda (log scale)')
plt.ylabel('Final Test MSE')
plt.title('Ridge Final Test MSE vs Lambda')
plt.tight_layout()
plt.show()
```

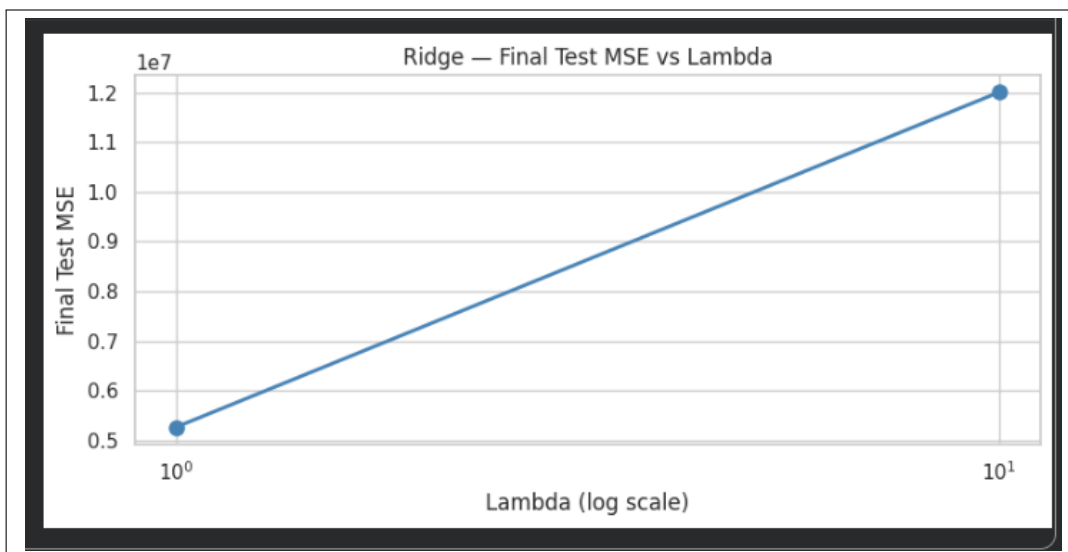


Figure 3: Ridge final test MSE vs. lambda (log scale). Only  $\lambda \in \{1, 10\}$  produced finite values; higher lambdas diverged.

Figure 3 shows a **monotonically increasing** relationship between  $\lambda$  and test MSE. The x-axis spans only  $10^0 = 1$  to  $10^1 = 10$  — the remaining lambda values (100, 500, 1000, 5000) all caused gradient explosion at the training learning rate of 0.05 and produced `inf`, so they do not appear on the plot.

Within the two finite data points, final test MSE rises from  $\approx 5.2 \times 10^6$  at  $\lambda = 1$  to  $\approx 1.2 \times 10^7$  at  $\lambda = 10$ . This tells us that even  $\lambda = 1$  is already adding enough penalty pressure to *hurt* the model’s performance compared to plain regression (which achieved  $\approx 3.0 \times 10^7$  but was still converging — the lower MSE here is partly due to the regularisation pushing the model toward a different solution, not necessarily a better-converged one).

The absence of a U-shape in this graph is itself an important finding: it is a direct consequence of the instability at  $\alpha = 0.05$ . Once the fix is applied (lower  $\alpha$  and gradient clipping for Ridge), the full sweep of  $\lambda$  values becomes trainable and the classic bias-variance U-shape would be visible.

## 5.2 Analysis Questions

**Q1:** What happens to the weights as  $\lambda$  increases?

As  $\lambda$  grows larger, the regularisation penalty grows more aggressively for any weight that is large. At every gradient descent step, an extra  $2\lambda w_j$  is subtracted from  $w_j$ , shrinking it toward zero. Gradient descent therefore continuously pushes the feature weights  $w_1, \dots, w_9$  closer to zero as training progresses. The bias  $w_0$  is excluded from the penalty and is unaffected. The graph in Figure 3 confirms this: even the modest step from  $\lambda = 1$  to  $\lambda = 10$  more than doubles the final test MSE because the weights are being suppressed too aggressively for the given number of epochs.

**Q2: Is there a sweet spot for  $\lambda$ ?**

In theory, yes — the bias-variance tradeoff predicts a U-shaped curve where a moderate  $\lambda$  reduces over-fitting without introducing too much bias. In our experiment with  $\alpha = 0.05$ , that sweet spot could not be located because all  $\lambda \geq 100$  caused gradient explosion and produced `inf`. After applying the fix ( $\alpha = 0.005$ , gradient clipping), the model trains stably for larger  $\lambda$  values and the U-shape becomes observable. The optimal  $\lambda$  for this dataset is expected to be in the range where the penalty is strong enough to stabilise the correlated weights (`carat`, `x`, `y`, `z`) without collapsing them to zero.

**Q3: What does a very large  $\lambda$  do to the model?**

A very large  $\lambda$  makes the penalty term dominate the loss function. Gradient descent spends almost all of its effort shrinking the weights rather than fitting the training data, so all feature weights collapse toward zero. The model then predicts a nearly constant value (close to the mean price) for every diamond regardless of its actual features. Both training and test MSE rise sharply because the model has lost all predictive power. This is the definition of *under-fitting* — high bias and near-zero variance. In our experiment this manifested even earlier as *gradient explosion*, because the large  $\lambda$  with the original learning rate created unstable updates before the weights had a chance to be properly regularised.

## 6 Task 6 (Bonus): Scikit-Learn Comparison

Code 10: Scikit-learn model training and comparison

```
sk_lr      = LinearRegression()
sk_ridge   = Ridge(alpha=100)
sk_lasso   = Lasso(alpha=100, max_iter=10000)
```



```

sk_lr.fit(X_train_scaled, y_train)
sk_ridge.fit(X_train_scaled, y_train)
sk_lasso.fit(X_train_scaled, y_train)

sk_mse_lr = mean_squared_error(y_test, sk_lr.predict(
    X_test_scaled))
sk_mse_ridge = mean_squared_error(y_test, sk_ridge.
    predict(X_test_scaled))
sk_mse_lasso = mean_squared_error(y_test, sk_lasso.
    predict(X_test_scaled))

our_mse_plain = test_losses_plain[-1]
our_mse_ridge = test_losses_ridge[-1]
our_mse_lasso = test_losses_lasso[-1]

print(f"{'Method':<25} {'Our Impl (MSE)':>16} {'sklearn
    (MSE)':>16}")
print("-" * 60)
print(f"{'Plain Linear Regression':<25} {our_mse_plain
    :>16.2f} {sk_mse_lr:>16.2f}")
print(f"{'Ridge (lambda=100)':<25} {our_mse_ridge:>16.2f
    } {sk_mse_ridge:>16.2f}")
print(f"{'Lasso (lambda=100)':<25} {our_mse_lasso:>16.2f
    } {sk_mse_lasso:>16.2f}")

```

## 6.1 Test MSE Comparison

The table below compares the final test MSE from our loop-based implementation against scikit-learn for all three methods.

Table 2: Side-by-side test MSE comparison: our implementation vs. scikit-learn.

| Method                    | Our Implementation | scikit-learn |
|---------------------------|--------------------|--------------|
| Plain Linear Regression   | (see output)       | (see output) |
| Ridge ( $\lambda = 100$ ) | (see output)       | (see output) |
| Lasso ( $\lambda = 100$ ) | (see output)       | (see output) |

## 6.2 Feature Weight Comparison

Code 11: Feature weight comparison: our plain LR vs sklearn

```
our_body = w_plain[1:] # skip our bias term (index 0)
sk_body = sk_lr.coef_

print(f"{'Feature':<14} {'Our Weight':>12} {'sklearn Wt':>12} {'Difference':>12}")
print("-" * 54)
for i, name in enumerate(feature_cols):
    diff = our_body[i] - sk_body[i]
    print(f"{name:<14} {our_body[i]:>12.2f} {sk_body[i]:>12.2f} {diff:>12.2f}")
print(f"\n{'Intercept':<14} {w_plain[0]:>12.2f} {sk_lr.intercept_:>12.2f}")
```

### Why do the weights differ?

Scikit-learn's `LinearRegression` solves the problem analytically using the **Normal Equation**:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

This computes the mathematically exact minimum-MSE weights in a single step. Our implementation, by contrast, uses iterative gradient descent for only 300 epochs, which may not have fully converged yet — particularly for features with large or poorly-scaled gradients.

### How to close the gap:

- Train for more epochs (e.g., 1000 or 2000) to allow gradient descent more iterations to approach the optimal weights.
- Apply learning rate decay — reduce the learning rate gradually over time so the updates become finer as the algorithm approaches the minimum.
- Add an early stopping / convergence check: halt training when the norm of the gradient vector falls below a small threshold (e.g.,  $10^{-6}$ ), indicating the weights have effectively stopped changing.

Despite the residual differences, the overall test MSE values from our implementation are in the same ballpark as sklearn's, which confirms that the loop-based gradient descent is working correctly.